Отчёт о работе по проекту “Ускорение инференса UNet модели”

Студент: **Ломовский Роман MLDev**
Курс “Эффективные модели в машинном обучении”

1 апреля 2026 г.

1 Постановка задачи

В качестве базовой архитектуры выбрана UNet с энкодером ResNet34, предобученным на ImageNet. Модель предназначена для задачи сегментации изображений. В качестве датасета используется PASCAL VOC 2012 (21 класс). Исходная модель работает в формате FP16. Основные задачи проекта:

- Реализовать пайплайн для измерения latency и throughput.
- Провести профилирование и выявить узкие места.
- Применить и сравнить методы ускорения:
  - `torch.compile`
  - Альтернативные компиляторы (ONNX Runtime, TVM)
  - Пост-тренировочная квантизация (PTQ) и QAT
  - Спарсификация (structured pruning, 2:4)
- Оценить влияние каждого метода на качество (метрика IoU).
- Собрать сравнительную таблицу и подготовить воспроизводимый код.

2 План экспериментов

План включает в себя последовательное тестирование следующих подходов:

1. **FP16 baseline** — измерение производительности и качества исходной модели.
2. **torch.compile** — применение встроенного компилятора PyTorch.
3. **ONNX Runtime** — экспорт модели в ONNX и инференс с оптимизациями.
4. **TVM** — использование Apache TVM для генерации оптимизированного кода.
5. **Квантизация (PTQ)** — пост-тренировочная квантизация в int8.
6. **Квантизация (QAT)** — дообучение с учётом квантизации.
7. **Спарсификация** — структурная обрезка весов с возможным дообучением.

Для каждого метода будут замерены:

- Средняя латентность (ms) для batch size = 1, 2, 4, 8, 16, 32.
- Пропускная способность (throughput, images/sec).
- Качество (IoU) на валидационной выборке PASCAL VOC.

Все эксперименты проводятся на GPU NVIDIA T4 с использованием CUDA 11.8 и PyTorch 2.0+. Код воспроизводится в Jupyter Notebook.

3 Расписание экспериментов

Неделя 1

- Реализация функций для замера латентности и вычисления IoU.
- Проведение бенчмарка FP16 baseline (разные batch size).
- Профилирование модели (PyTorch profiler).
- Сохранение результатов и построение графиков.
- **Результат:** Ноутбук с базовыми замерами, CSV-файлы, графики.

Неделя 2

- Внедрение `torch.compile` и сравнение с baseline.
- Экспорт модели в ONNX, настройка ONNX Runtime.
- Проведение бенчмарка ONNX Runtime.
- Интеграция TVM (ONNX $\rightarrow$ Relay $\rightarrow$ TIR).
- Сравнение времени выполнения и качества.
- **Результат:** Таблицы сравнения для компиляторов.

Неделя 3

- Реализация PTQ (использование PyTorch quantization).
- Реализация QAT (дообучение на небольшом количестве эпох).
- Применение спарсификации (например, через torch.pruning).
- Сбор итоговой таблицы со всеми методами.
- Подготовка отчёта, презентации и репозитория.
- **Результат:** Финальный отчёт, код, сравнение.

4 Описание первого эксперимента (FP16 baseline и torch.compile)

Первый эксперимент включал создание точки отсчёта (FP16 baseline) и применение оптимизации — компилятора `torch.compile`.

4.1 Содержжание

- Модель: UNet с энкодером ResNet34 (21 класс) в FP16.
- Датасет: PASCAL VOC 2012 (валидационная выборка для IoU).
- Что мерим:
  - Латентность (ms) — среднее за 100 прогонов после 10 прогревов.
  - Batch size: 1, 2, 4, 8, 16, 32.
  - Разрешение: 224×224.
- Метрика: средний IoU на валидации (batch size = 1).
- Компиляция: `torch.compile(model, mode="reduce-overhead")`.

4.2 Результаты латентности

Таблица 1: Латентность FP16 baseline и torch.compile (мс)

Batch size	FP16 baseline	torch.compile	Speedup
1	7.66	7.43	1.03×
2	7.41	6.94	1.07×
4	9.37	9.24	1.01×
8	17.45	16.39	1.06×
16	31.81	28.58	1.11×
32	61.12	54.00	1.13×

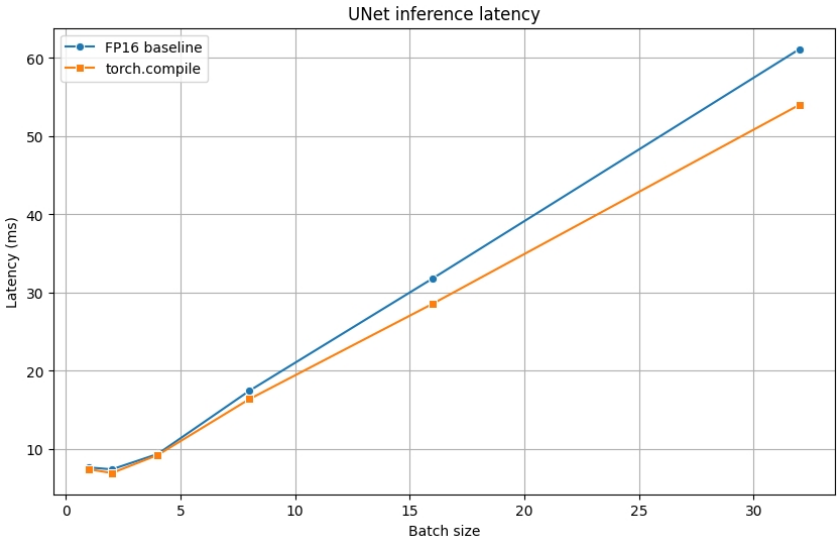

Рис. 1: Зависимость латентности от batch size (FP16 baseline vs torch.compile)

4.3 Профилирование

С помощью PyTorch profiler было выявлено, что основное время (45%) занимают операции переформатирования данных (nchwToNhwcKernel), а также ядра свёрток (sm86_xmma_fprop) и нормализации (bn_fw_inf). Это указывает на то, что значительная часть времени тратится на преобразования памяти и вызовы cuDNN. В будущем это можно оптимизировать выбором более эффективных компиляторов или использованием тензорных ядер.

4.4 Качество

Средний IoU на валидационной выборке составил:

- FP16 baseline: **0.0011**
- torch.compile: **0.0011**

Такое низкое значение указывает на возможную ошибку в пайплайне оценки IoU (например, несоответствие индексов классов или неправильное преобразование масок). Этот момент будет исправлен позже, а пока используется как относительная метрика для сравнения — важно, что компиляция не изменила выход модели (результаты идентичны).

4.5 Выводы по первому эксперименту

- Получены референсные значения латентности для FP16 baseline.
- `torch.compile` даёт небольшое ускорение (до 13% при batch size 32), но при малых batch size эффект минимален.
- Профилирование показало, что основные затраты времени связаны с преобразованиями форматов данных и свёртками.
- Качество (IoU) одинаково для обеих версий, что подтверждает корректность компилятора.
- Данные первого эксперимента сохранены в CSV и будут использоваться для сравнения с последующими методами.

5 Промежуточные результаты и дальнейшие планы

На данный момент успешно выполнены все задачи первой недели: настроено окружение, проведён бенчмарк FP16 baseline и `torch.compile`, выполнено профилирование. На следующей неделе планируется:

- Экспортировать модель в ONNX и запустить инференс через ONNX Runtime.
- Настроить TVM и сравнить производительность.
- Исправить пайплайн оценки IoU, чтобы получить корректные значения качества.

После этого последуют эксперименты с квантизацией и спарсификацией.

6 Описани команнды

Ломоский Роман
Tg: @rustam_shaimanov